

```
---
title: "Color-clustering con GMM en imágenes RGB – ejemplo mínimo (EM a mano)"
author: "FCFM-UNACH | Matemáticas Aplicadas"
date: "2025-08-27"
output:
  html_document:
    toc: true
    toc_float: true
    df_print: paged
  pdf_document: default
fontsize: 11pt
---
```

Motivación breve

Al comprimir colores en una imagen o separar **regiones cromáticas** (cielo, vegetación, paredes, etc.), una estrategia simple y efectiva es modelar los **colores de píxel** $((R,G,B))$ como una **mezcla de gaussianas** (GMM). Cada **componente** (media y covarianza) describe un “color promedio + variación” (una **nube** en el cubo RGB). Con **EM** aprendemos:

- **pesos** (π_k) (proporciones),
- **medias** $(\mu_k \in \mathbb{R}^3)$,
- **covarianzas** $(\Sigma_k \in \mathbb{R}^{3 \times 3})$.

Luego podemos:

- **Cuantizar** la imagen: reemplazar cada píxel por la **media** de su componente más probable (MAP).
- Construir la **posterior mean image**: $\text{color} = \sum_k \gamma_{ik} \mu_k$, que usa **responsabilidades blandas** (γ_{ik}) .

Formulación (3D, RGB) y EM

Sea $(X \in \mathbb{R}^3)$ el vector normalizado $((R,G,B) \in [0,1]^3)$. Modelo:

$$p(x|\Theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k), \quad \pi_k > 0, \sum_k \pi_k = 1.$$

Variables latentes $(Z_i \in \{1, \dots, K\})$. **E-step:**

$$\gamma_{ik} = \frac{\pi_k \phi_3(x_i; \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \phi_3(x_i; \mu_j, \Sigma_j)}.$$

M-step:

$$N_k = \sum_i \gamma_{ik}, \quad \pi_k^{\text{new}} = \frac{N_k}{N}, \quad \mu_k^{\text{new}} = \frac{1}{N_k} \sum_i \gamma_{ik} x_i, \quad \Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_i \gamma_{ik} (x_i - \mu_k^{\text{new}})(x_i - \mu_k^{\text{new}})^{\text{top}}.$$

Para el **cálculo manual** usaremos (Σ_k) **diagonal** (independencia entre canales dentro del componente), lo que simplifica (ϕ_3) a producto de normales 1D.

Tabla mínima (6 píxeles) para una vuelta de EM a mano

Elegimos 6 “píxeles” (RGB en $[0,255]$) representativos de **tres colores** (rojo, verde, azul) con pequeñas variaciones. Convertimos a $([0,1])$.

```
RGB_255 <- matrix(c(
  230, 30, 35, # rojo 1
  240, 35, 40, # rojo 2
  20, 220, 25, # verde 1 (frontera)
  25, 210, 35, # verde 2
  30, 35, 230, # azul 1 (frontera)
  28, 40, 240 # azul 2
), ncol = 3, byrow = TRUE)
colnames(RGB_255) <- c("R", "G", "B")

X <- RGB_255/255 # normalizado [0,1]
DF <- data.frame(i = 1:nrow(X), round(X, 4))
DF
```

```
##   i    R    G    B
## 1 1 0.9020 0.1176 0.1373
## 2 2 0.9412 0.1373 0.1569
## 3 3 0.0784 0.8627 0.0980
## 4 4 0.0980 0.8235 0.1373
## 5 5 0.1176 0.1373 0.9020
## 6 6 0.1098 0.1569 0.9412
```

Inicialización (intuitiva, $K=3$, diagonal):

- Pesos: $(\pi_k = 1/3)$.

- Medias (en [0,1]): $\mu_1=(0.9,0.1,0.1)$ rojo, $\mu_2=(0.1,0.9,0.1)$ verde, $\mu_3=(0.1,0.1,0.9)$ azul.
- Desv. estándar por canal (suave): $\sigma_k=(0.08,0.08,0.08)$ para todos.

```
K <- 3
pi <- rep(1/K, K)
mu <- rbind(c(0.9, 0.1, 0.1),
            c(0.1, 0.9, 0.1),
            c(0.1, 0.1, 0.9))
sd <- rbind(c(0.08, 0.08, 0.08),
            c(0.08, 0.08, 0.08),
            c(0.08, 0.08, 0.08))
```

E-step (una vuelta)

Densidad 3D diagonal: $\phi_3(x; \mu, \Sigma) = \prod_{c \in \{R, G, B\}} \phi(x^{(c)}; \mu^{(c)}, \sigma^{(c)})$.

```
d3_diag <- function(x, mu, sd){
  dnorm(x[1], mu[1], sd[1]) *
  dnorm(x[2], mu[2], sd[2]) *
  dnorm(x[3], mu[3], sd[3])
}

dens <- sapply(1:K, function(k) apply(X, 1, function(x) d3_diag(x, mu[k,], sd[k,])))
num <- sweep(dens, 2, pi, `*`) # multiplicar columnas por pesos
den <- rowSums(num)
Gamma <- num / den           # responsabilidades (N x K)
Gamma_round <- round(Gamma, 4)
cbind(DF, Gamma_round)
```

```
##      i      R      G      B 1 2 3
## 1 1 0.9020 0.1176 0.1373 1 0 0
## 2 2 0.9412 0.1373 0.1569 1 0 0
## 3 3 0.0784 0.8627 0.0980 0 1 0
## 4 4 0.0980 0.8235 0.1373 0 1 0
## 5 5 0.1176 0.1373 0.9020 0 0 1
## 6 6 0.1098 0.1569 0.9412 0 0 1
```

Deberías ver responsabilidades cercanas a 1 para cada color típico y valores intermedios (~frontera) en los dos puntos “dudosos”.

M-step (actualización)

```
Nk <- colSums(Gamma)
pi_n <- Nk / nrow(X)
mu_n <- t(Gamma) %*% X / Nk           # (K x 3)
# varianzas diagonales por canal
var_n <- matrix(0, nrow=K, ncol=3)
for(k in 1:K){
  Xc <- sweep(X, 2, mu_n[k,], `-`)
  var_n[k,] <- colSums(Gamma[,k] * Xc^2)/Nk[k]
}
sd_n <- sqrt(var_n)

Ms <- data.frame(
  componente = paste0(1:K),
  pi_k = pi_n,
  mu_R = mu_n[,1], mu_G = mu_n[,2], mu_B = mu_n[,3],
  sd_R = sd_n[,1], sd_G = sd_n[,2], sd_B = sd_n[,3],
  stringsAsFactors = FALSE
)
nums <- sapply(Ms, is.numeric); Ms_round <- Ms
Ms_round[,nums] <- lapply(Ms_round[,nums], function(z) round(z, 4))
Ms_round
```

```
##  componente  pi_k  mu_R  mu_G  mu_B  sd_R  sd_G  sd_B
## 1           1 0.3333 0.9216 0.1275 0.1471 0.0196 0.0098 0.0098
## 2           2 0.3333 0.0882 0.8431 0.1176 0.0098 0.0196 0.0196
## 3           3 0.3333 0.1137 0.1471 0.9216 0.0039 0.0098 0.0196
```

Log-verosimilitud (sube tras EM)

```
loglik <- function(X, pi, mu, sd){
  dens <- sapply(1:length(pi), function(k) apply(X, 1, function(x) d3_diag(x, mu[k,], sd[k,])))
  sum(log(rowSums(sweep(dens, 2, pi, `*`))))
}
ll0 <- loglik(X, pi, mu, sd)
ll1 <- loglik(X, pi_n, mu_n, sd_n)
c(ll_inicial = round(ll0, 4), ll_despues_EM = round(ll1, 4))
```

```
##      ll_inicial ll_despues_EM
##      20.4677      47.4043
```

De la tabla mínima a una imagen sintética

Generamos una **imagen sintética** (alto $\times$ ancho $\times$ 3) con 3 regiones cromáticas + ruido suave, para visualizar cuantización.

```
set.seed(7)
H <- 80; W <- 120
img <- array(0, dim = c(H, W, 3))

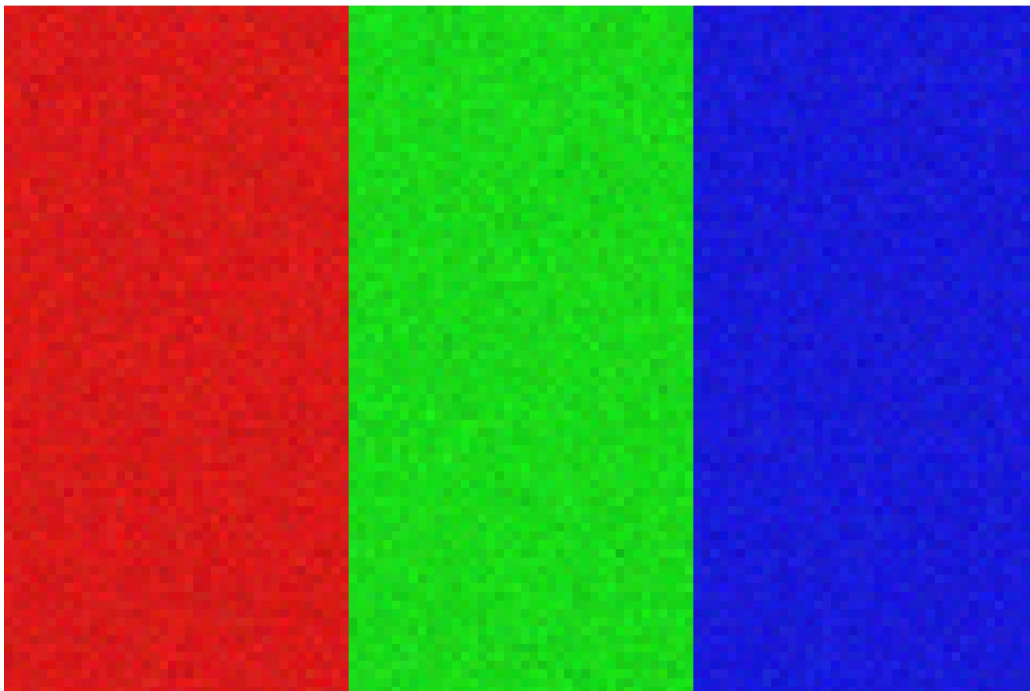
# tres bloques de color (R, G, B) con leve ruido
img[, 1:(W/3), 1] <- 0.85 + rnorm(H*W/3, 0, 0.03)
img[, 1:(W/3), 2] <- 0.10 + rnorm(H*W/3, 0, 0.02)
img[, 1:(W/3), 3] <- 0.10 + rnorm(H*W/3, 0, 0.02)

img[, (W/3+1):(2*W/3), 1] <- 0.10 + rnorm(H*W/3, 0, 0.02)
img[, (W/3+1):(2*W/3), 2] <- 0.85 + rnorm(H*W/3, 0, 0.03)
img[, (W/3+1):(2*W/3), 3] <- 0.10 + rnorm(H*W/3, 0, 0.02)

img[, (2*W/3+1):W, 1] <- 0.10 + rnorm(H*W/3, 0, 0.02)
img[, (2*W/3+1):W, 2] <- 0.10 + rnorm(H*W/3, 0, 0.02)
img[, (2*W/3+1):W, 3] <- 0.85 + rnorm(H*W/3, 0, 0.03)

# recorte a [0,1]
img[img<0] <- 0; img[img>1] <- 1

# muestra
grid::grid.raster(img, interpolate = FALSE)
```



Convertimos la imagen a una matriz $(N \times 3)$ (píxeles por filas, RGB por columnas).

```
Ximg <- matrix(img, ncol = 3) # (H*W) x 3
dim(Ximg)
```

```
## [1] 9600    3
```

Ajuste con mclust (G=3..6) y posterior mean image

Ajustamos un GMM y generamos dos versiones de la imagen:

- **MAP image:** μ_k del componente con mayor γ_{ik} .
- **Posterior mean image:** $\sum_k \gamma_{ik} \mu_k$.

```
# install.packages("mclust") # si hace falta
library(mclust)
```

```
## Package 'mclust' version 6.1.1
## Type 'citation("mclust")' for citing this R package in publications.
```

```
fit <- Mclust(Ximg, G = 3:6) # selección automática de K por BIC
summary(fit)
```

```
## -----
## Gaussian finite mixture model fitted by EM algorithm
## -----
##
## Mclust EVI (diagonal, equal volume, varying shape) model with 3 components:
##
## log-likelihood    n df      BIC      ICL
##      57193.88 9600 18 114222.7 114222.7
##
## Clustering table:
##      1      2      3
## 3200 3200 3200
```

```
K_hat <- fit$G
cat("Número de componentes elegido (BIC):", K_hat, "\n")
```

```
## Número de componentes elegido (BIC): 3
```

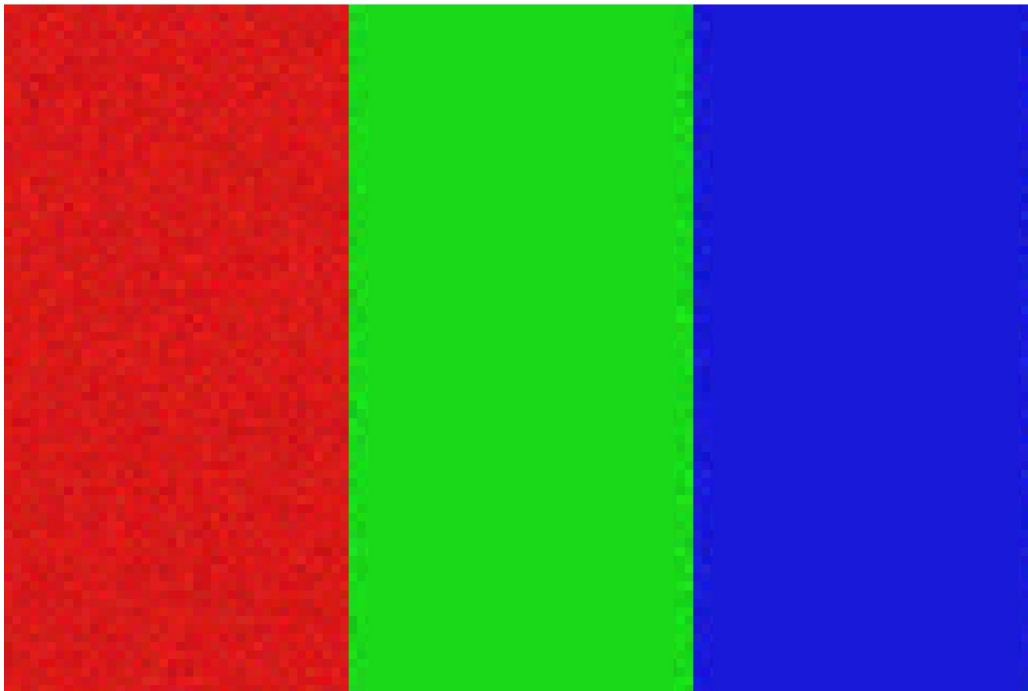
```
# responsabilidades (N x K) y medias (3 x K)
Z <- fit$z
MU <- t(fit$parameters$mean) # K x 3

# (1) MAP (asignación dura)
k_map <- max.col(Z) # argmax por fila
X_map <- MU[k_map, ] # N x 3
img_map <- array(X_map, dim = c(H, W, 3))

# (2) Posterior mean (asignación blanda)
X_soft <- Z %*% MU # N x 3
img_soft <- array(X_soft, dim = c(H, W, 3))
```

Visualización

```
par(mfrow = c(1,3), mar = c(1,1,2,1))
plot.new(); grid::grid.raster(img, interpolate = FALSE); title("Original")
plot.new(); grid::grid.raster(img_map, interpolate = FALSE); title("MAP (cuantización dura)")
plot.new(); grid::grid.raster(img_soft, interpolate = FALSE); title("Posterior mean (blanda)")
```

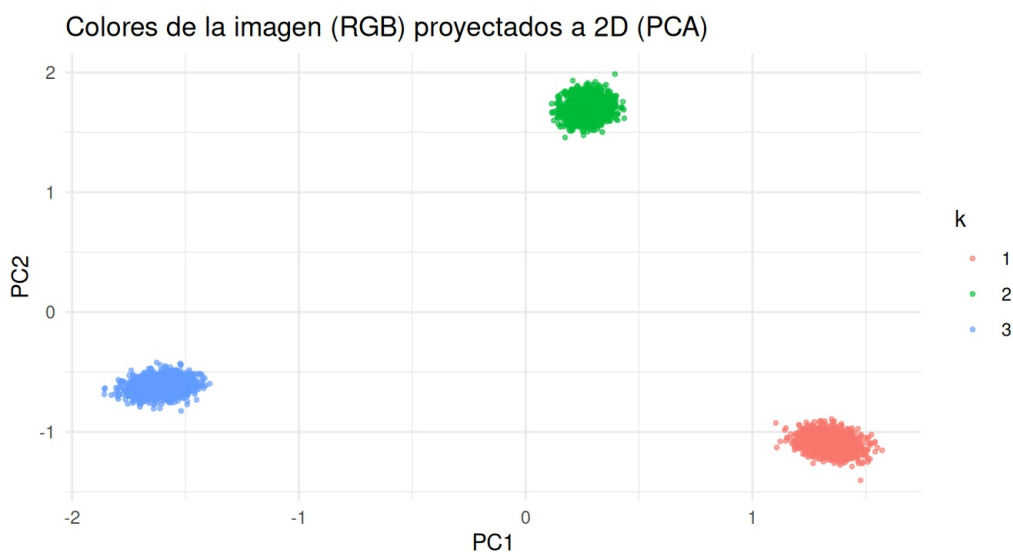


```
par(mfrow = c(1,1))
```

Proyección a 2D (PCA) y coloreo por componente

Para entender la **geometría de colores** en el cubo RGB, proyectamos a 2D con PCA y coloreamos por la etiqueta MAP.

```
pc <- prcomp(Ximg, center = TRUE, scale. = TRUE)
dfp <- data.frame(PC1 = pc$x[,1], PC2 = pc$x[,2], k = factor(k_map))
# submuestreo para graficar rápido
set.seed(1)
idx <- sample(1:nrow(dfp), min(4000, nrow(dfp)))
ggplot2::ggplot(dfp[idx,], ggplot2::aes(PC1, PC2, color = k)) +
  ggplot2::geom_point(alpha = 0.6, size = 0.7) +
  ggplot2::labs(title = "Colores de la imagen (RGB) proyectados a 2D (PCA)") +
  ggplot2::theme_minimal()
```



(Opcional) Cargar una imagen real

Si quieres probar con una imagen **PNG** o **JPEG**, descomenta y ajusta:

```
# install.packages(c("png","jpeg"))
library(png); library(jpeg)
img_path <- "mi_imagen.png"      # o .jpg
if(grepl("\\.png$", img_path, ignore.case=TRUE)){
  img_real <- png::readPNG(img_path)      # [0,1]
} else {
  img_real <- jpeg::readJPEG(img_path)    # [0,1]
}
H2 <- dim(img_real)[1]; W2 <- dim(img_real)[2]
X2 <- matrix(img_real, ncol = 3)

fit2 <- Mclust(X2, G = 3:6)
Z2 <- fit2$z
MU2 <- t(fit2$parameters$mean)

k2_map <- max.col(Z2)
img2_map <- array(MU2[k2_map, ], dim = c(H2, W2, 3))
img2_soft <- array(Z2 %*% MU2, dim = c(H2, W2, 3))

par(mfrow = c(1,3), mar = c(1,1,2,1))
plot.new(); grid::grid.raster(img_real); title("Original")
plot.new(); grid::grid.raster(img2_map); title("MAP")
plot.new(); grid::grid.raster(img2_soft); title("Posterior mean")
par(mfrow = c(1,1))
```

Ejercicios sugeridos

1. **Repite el E-step** con $(\pi^{\text{new}}, \mu^{\text{new}}, \Sigma^{\text{new}})$ para la tabla de 6 píxeles y verifica que la **log-verosimilitud** vuelve a subir.
2. Cambia la **inicialización** (pesos y medias) y observa cómo se reparte la responsabilidad en los dos píxeles frontera.
3. En la imagen sintética, limita el modelo a **covarianzas diagonales** (`modelNames="EEE", "VVV", "EEE"` variantes) y compara **BIC** con covarianzas completas.
4. Ajusta (G) manualmente (p.ej. 3, 4, 5, 6) y compara **PSNR** entre original y “posterior mean image”.
5. Prueba con una **imagen real** (bloque opcional) y discute qué representan las medias (μ_k) aprendidas (paleta de colores dominante).

Notas técnicas y pequeñas precauciones

- La **tabla mínima** usa (Σ) **diagonal** para que el cálculo manual sea directo. En práctica, `Mclust` suele elegir modelos con covarianza **completa** (más flexibles).
- Si observas advertencias por valores fuera de $[0,1]$ tras añadir ruido, ya los **recortamos**.
- Si tu versión de `ggplot2` es antigua, cambia `alpha/size` según necesidad; evitamos `linewidth` para compatibilidad.